CATEGORY 5 COMPUTATION - IRIS DATASET

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay
from sklearn.preprocessing import StandardScaler
```

```
# Load the Iris dataset
iris = load_iris()
X, y = iris.data, iris.target
target_names = iris.target_names
```

```
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

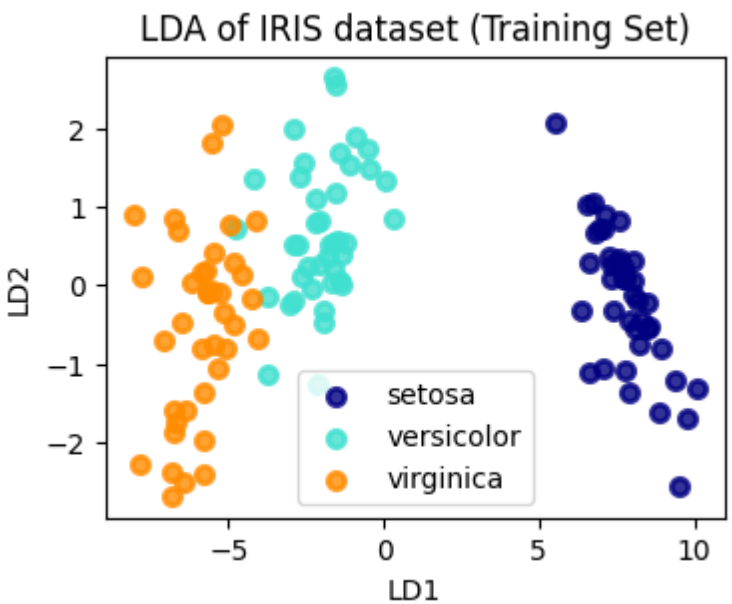
```
 #Standardize the features (Important for LDA as it uses covariance)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
# Create an LDA model for classification
lda_clf = LinearDiscriminantAnalysis()
lda_clf.fit(X_train_scaled, y_train)
```

```
▾ LinearDiscriminantAnalysis
LinearDiscriminantAnalysis()
```

```
# Use LDA for dimensionality reduction (to 2D for plotting)
lda_for_plot = LinearDiscriminantAnalysis(n_components=2)
X_train_lda = lda_for_plot.fit_transform(X_train_scaled, y_train)
```

```
# Plot the transformed training data
plt.figure(figsize=(4, 3))
colors = ['navy', 'turquoise', 'darkorange']
lw = 2

for color, i, target_name in zip(colors, [0, 1, 2], target_names):
    plt.scatter(X_train_lda[y_train == i, 0],
                X_train_lda[y_train == i, 1],
                color=color, alpha=.8, lw=lw, label=target_name)
plt.legend(loc='best', shadow=False, scatterpoints=1)
plt.title('LDA of IRIS dataset (Training Set)')
plt.xlabel('LD1')
plt.ylabel('LD2')
plt.show()
```

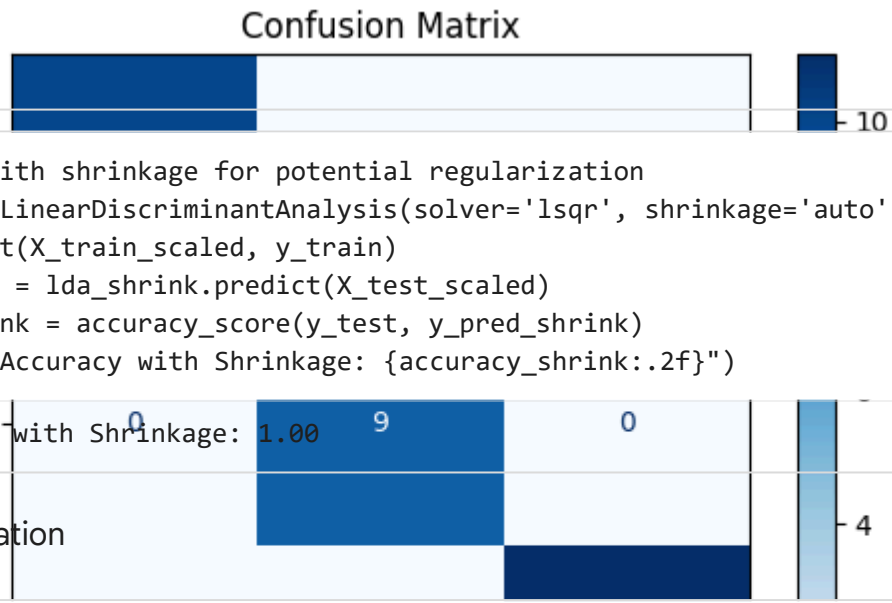


```
# Predict on the test set
y_pred = lda_clf.predict(X_test_scaled)
```

```
# Calculate and print accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Test Accuracy: {accuracy:.2f}")
```

```
Test Accuracy: 1.00
```

```
# Display the confusion matrix
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=target_names)
disp.plot(cmap=plt.cm.Blues)
plt.title("Confusion Matrix")
plt.show()
```




```
# Using LDA with shrinkage for potential regularization
lda_shrink = LinearDiscriminantAnalysis(solver='lsqr', shrinkage='auto')
lda_shrink.fit(X_train_scaled, y_train)
y_pred_shrink = lda_shrink.predict(X_test_scaled)
accuracy_shrink = accuracy_score(y_test, y_pred_shrink)
print(f"Test Accuracy with Shrinkage: {accuracy_shrink:.2f}")
```

```
Test Accuracy with Shrinkage: 1.00
```

Category 5 - Illustration

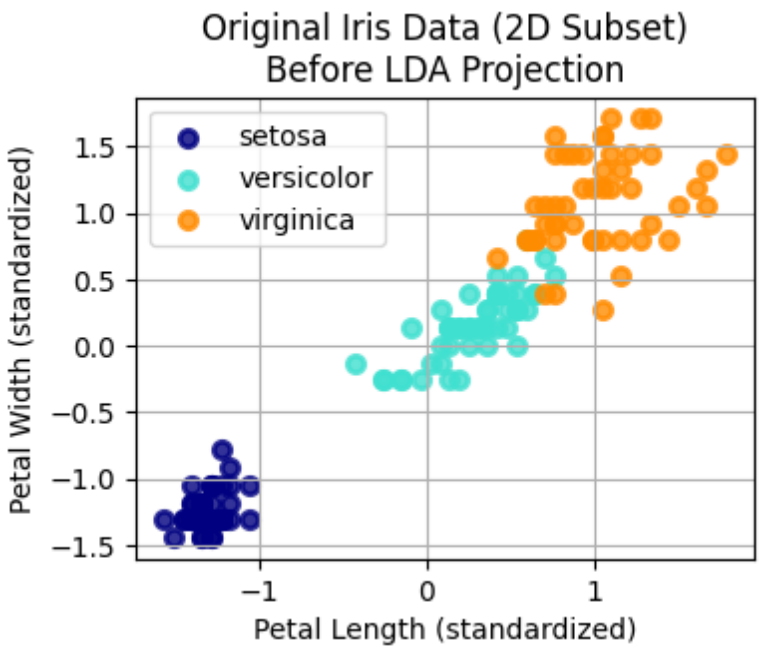
```
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
```

```
# Load Iris dataset
iris = datasets.load_iris()
X = iris.data
y = iris.target
target_names = iris.target_names
```

```
# For illustration, use only two features: Petal length and Petal width
X_2d = X[:, 2:4]
```

```
# Standardize the data
scaler = StandardScaler()
X_2d_scaled = scaler.fit_transform(X_2d)
```

```
# Plotting the original data
plt.figure(figsize=(4, 3))
colors = ['navy', 'turquoise', 'darkorange']

for color, i, target_name in zip(colors, [0, 1, 2], target_names):
    plt.scatter(X_2d_scaled[y == i, 0],
                X_2d_scaled[y == i, 1],
                color=color, alpha=.8, lw=2, label=target_name)

plt.legend(loc='best', shadow=False)
plt.title('Original Iris Data (2D Subset)\nBefore LDA Projection')
plt.xlabel('Petal Length (standardized)')
plt.ylabel('Petal Width (standardized)')
plt.grid(True)
plt.show()
```

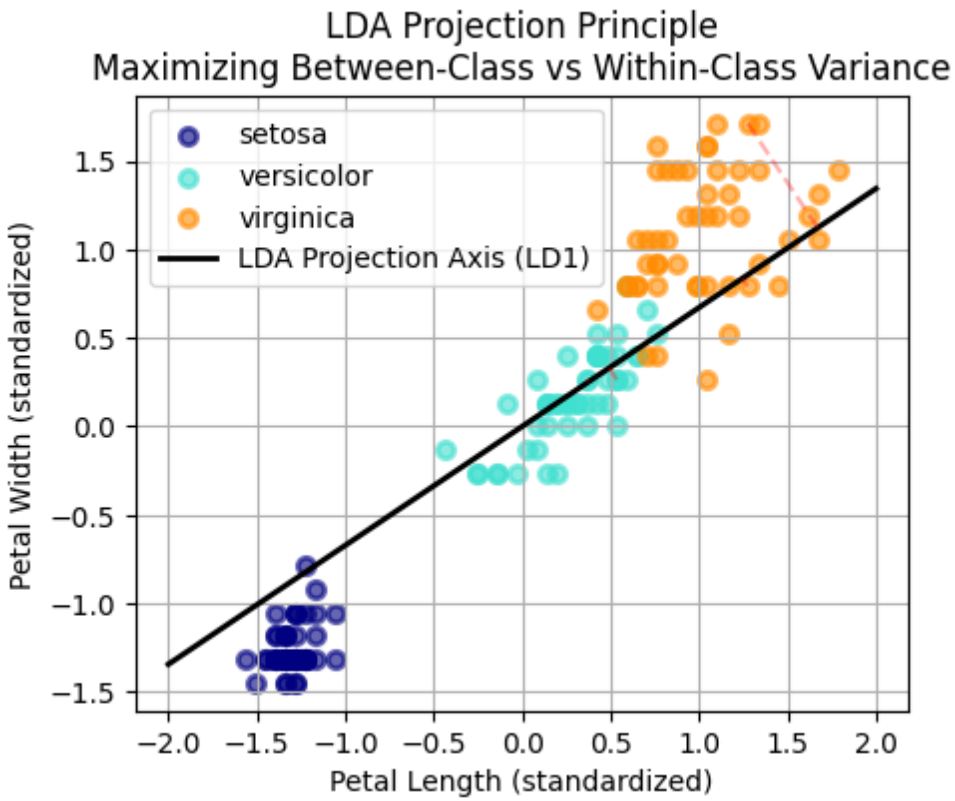



```
import numpy as np
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
```

```
# Apply LDA to project 2D data to 1D
lda = LinearDiscriminantAnalysis(n_components=1)
X_lda_1d = lda.fit_transform(X_2d_scaled, y)
```

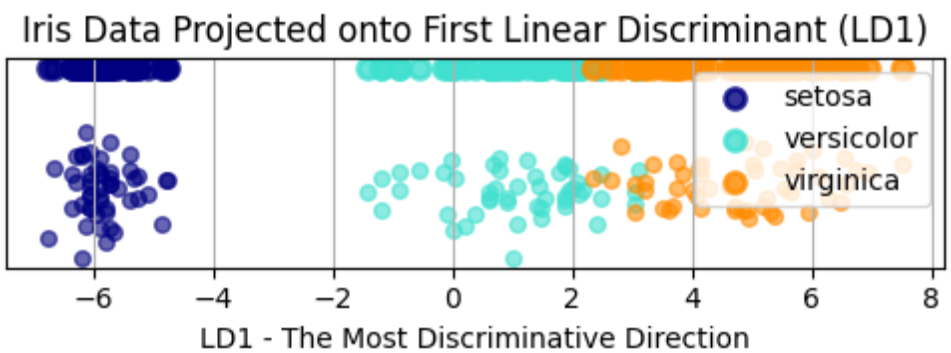
```
# Create a visualization showing the projection
plt.figure(figsize=(5, 4))

# Plot original data
for color, i, target_name in zip(colors, [0, 1, 2], target_names):
    plt.scatter(X_2d_scaled[y == i, 0],
                X_2d_scaled[y == i, 1],
                color=color, alpha=.6, lw=2, label=target_name)
# Plot the LDA projection direction (the discriminant vector)
lda_direction = lda.scalings_[:, 0]  # The first (and only) discriminant

# Create a line representing the LDA direction
x_plot = np.linspace(-2, 2, 100)
y_plot = (lda_direction[1] / lda_direction[0]) * x_plot

# Plot the LDA projection line
plt.plot(x_plot, y_plot, 'k-', linewidth=2, label='LDA Projection Axis (LD1)')
# Add projection lines for a few points
for i in [50, 75, 100, 125]:  # Sample points from different classes
    x_orig = X_2d_scaled[i, 0]
    y_orig = X_2d_scaled[i, 1]

    # Project point onto LDA direction
    t = (lda_direction[0]*x_orig + lda_direction[1]*y_orig) / (lda_direction[0]**2 + lda_direction[1]**2)
    x_proj = t * lda_direction[0]
    y_proj = t * lda_direction[1]

    # Draw projection line
    plt.plot([x_orig, x_proj], [y_orig, y_proj], 'r--', alpha=0.3)
plt.legend(loc='best')
plt.title('LDA Projection Principle\nMaximizing Between-Class vs Within-Class Variance')
plt.xlabel('Petal Length (standardized)')
plt.ylabel('Petal Width (standardized)')
plt.grid(True)
```

```
plt.axis('equal')
plt.show()
```



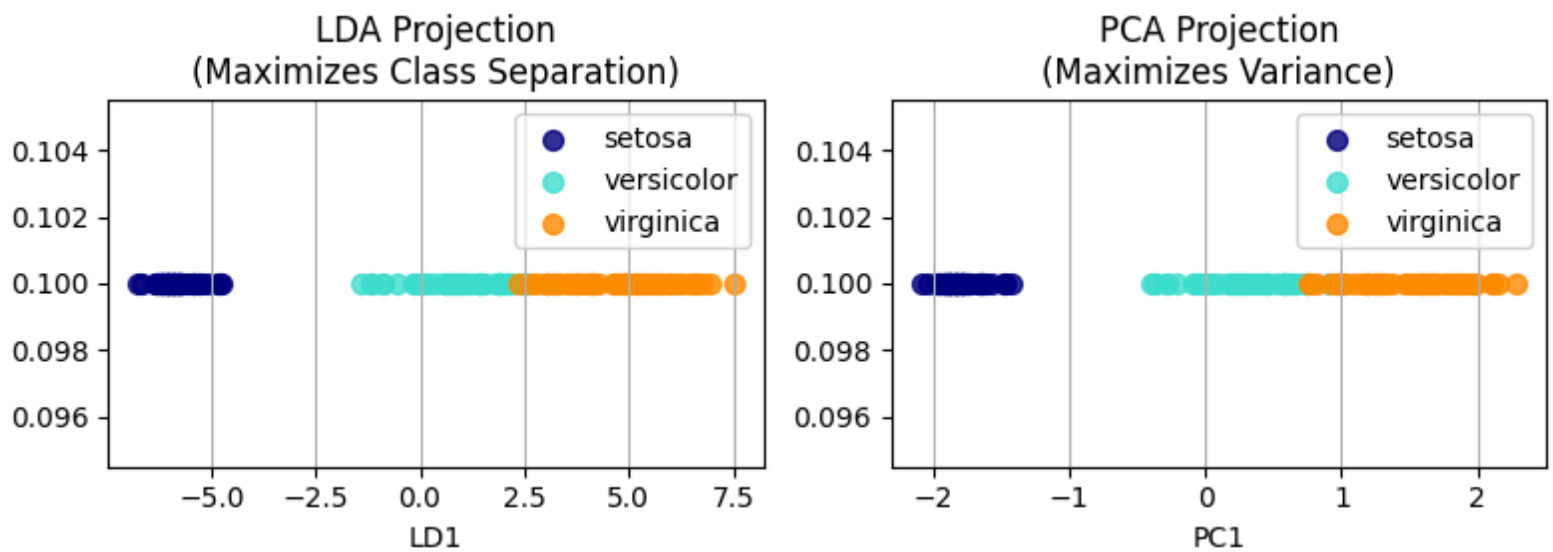

```
plt.figure(figsize=(5, 2))

# Create the 1D projection plot
for color, i, target_name in zip(colors, [0, 1, 2], target_names):
    class_data = X_lda_1d[y == i]
    plt.scatter(class_data, np.zeros_like(class_data) + 0.1,
                color=color, alpha=.8, lw=2, label=target_name, s=50)

# Add some jitter to see density better
y_jitter = np.random.normal(0, 0.02, size=len(X_lda_1d))
for color, i in zip(colors, [0, 1, 2]):
    class_data = X_lda_1d[y == i]
    jitter = y_jitter[y == i]
```

```
    jitter = y_jitter[y == i]
    plt.scatter(class_data, jitter, color=color, alpha=0.6, s=30)

plt.yticks([])
plt.title('Iris Data Projected onto First Linear Discriminant (LD1)')
plt.xlabel('LD1 - The Most Discriminative Direction')
plt.legend()
plt.grid(True, axis='x')
plt.tight_layout()
plt.show()
```




```
from sklearn.decomposition import PCA

# Apply PCA for comparison
pca = PCA(n_components=1)
X_pca_1d = pca.fit_transform(X_2d_scaled)

# Create comparison plot
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(8, 3))

# LDA projection
for color, i, target_name in zip(colors, [0, 1, 2], target_names):
    ax1.scatter(X_lda_1d[y == i], np.zeros_like(X_lda_1d[y == i]) + 0.1,
                color=color, alpha=.8, s=50, label=target_name)
ax1.set_title('LDA Projection\n(Maximizes Class Separation)')
ax1.set_xlabel('LD1')
ax1.grid(True, axis='x')
ax1.legend()
```

```
# PCA projection
for color, i, target_name in zip(colors, [0, 1, 2], target_names):
    ax2.scatter(X_pca_1d[y == i], np.zeros_like(X_pca_1d[y == i]) + 0.1,
                color=color, alpha=.8, s=50, label=target_name)
ax2.set_title('PCA Projection\n(Maximizes Variance)')
ax2.set_xlabel('PC1')
ax2.grid(True, axis='x')
ax2.legend()

plt.tight_layout()
plt.show()
```



Hyperparameter tuning for the Iris dataset

```
from sklearn.model_selection import GridSearchCV
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

# Define the parameter grid
param_grid = {
    'solver': ['svd', 'lsqr', 'eigen'],
```

```
    'shrinkage': [None, 'auto', 0.2, 0.5, 0.8],
    'priors': [None, [0.33, 0.33, 0.33], [0.5, 0.25, 0.25]]  # Custom priors
}

# Create LDA classifier
lda = LinearDiscriminantAnalysis()

# Perform grid search with 5-fold cross-validation
grid_search = GridSearchCV(lda, param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid_search.fit(X_train_scaled, y_train)

# Display the best parameters and score
print("Best Parameters:", grid_search.best_params_)
print("Best Cross-Validation Score:", grid_search.best_score_)

# Evaluate on test set
best_lda = grid_search.best_estimator_
y_pred_best = best_lda.predict(X_test_scaled)
test_accuracy = accuracy_score(y_test, y_pred_best)
print(f"Test Accuracy with Best Model: {test_accuracy:.3f}")
```

```
Best Parameters: {'priors': None, 'shrinkage': None, 'solver': 'svd'}
Best Cross-Validation Score: 0.975
Test Accuracy with Best Model: 1.000
/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_validation.py:528: FitFailedWarning:
60 fits failed out of a total of 225.
The score on these train-test partitions for these parameters will be set to nan.
If these failures are not expected, you can try to debug them by setting error_score='raise'.

Below are more details about the failures:
--------------------------------------------------------------------------------
60 fits failed with the following error:
Traceback (most recent call last):
  File "/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_validation.py", line 866, in _fit_and_sco
    estimator.fit(X_train, y_train, **fit_params)
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line 1389, in wrapper
    return fit_method(estimator, *args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/sklearn/discriminant_analysis.py", line 700, in fit
    raise NotImplementedError("shrinkage not supported with 'svd' solver.")
NotImplementedError: shrinkage not supported with 'svd' solver.
```

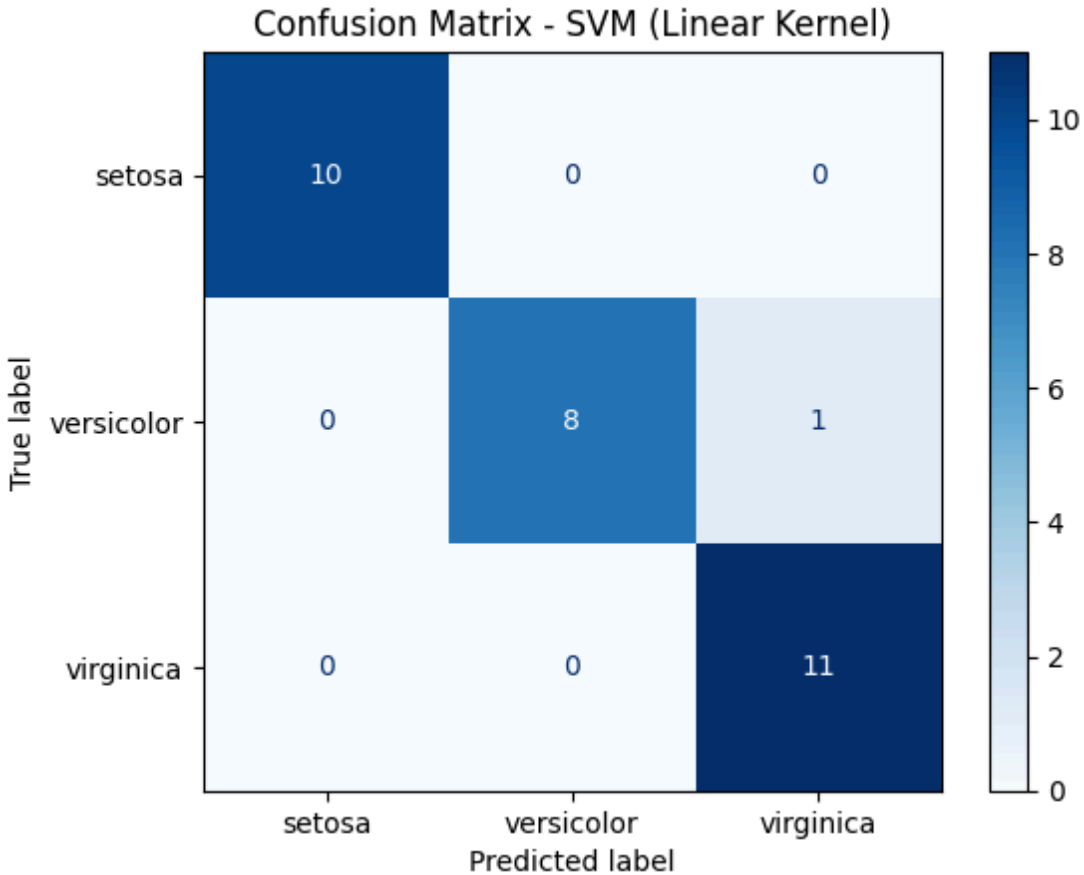
```
  warnings.warn(some_fits_failed_message, FitFailedWarning)
/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_search.py:1108: UserWarning: One or more of the t
        nan 0.95833333 0.95833333        nan 0.925      0.925
        nan 0.86666667 0.86666667 0.975      0.975      0.975
        nan 0.96666667 0.96666667        nan 0.95833333 0.95833333
        nan 0.93333333 0.93333333        nan 0.875      0.875
 0.975      0.96666667 0.96666667        nan 0.95833333 0.95833333
        nan 0.95833333 0.95833333        nan 0.925      0.925
        nan 0.875      0.875     ]
  warnings.warn(
```

Category 6:

SVM

```
# Applying SVM for Classification (using same iris dataset)

from sklearn.svm import SVC

svm_clf = SVC(kernel='linear', random_state=42)

# Train the SVM model
svm_clf.fit(X_train_scaled, y_train)

# Predict on the test set
y_pred_svm = svm_clf.predict(X_test_scaled)

# Calculate and print accuracy
accuracy_svm = accuracy_score(y_test, y_pred_svm)
print(f"Test Accuracy (SVM - Linear Kernel): {accuracy_svm:.2f}")

# Display the confusion matrix
cm_svm = confusion_matrix(y_test, y_pred_svm)
disp_svm = ConfusionMatrixDisplay(confusion_matrix=cm_svm, display_labels=target_names)
disp_svm.plot(cmap=plt.cm.Blues)
plt.title("Confusion Matrix - SVM (Linear Kernel)")
plt.show()
```

```
Test Accuracy (SVM - Linear Kernel): 0.97
```



Category 7:

Neural Networks

```
# Dataset: Telco Customer Churn
# Goal: Predict whether a customer will churn using a Neural Network

# 1. Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.layers import Dense, Dropout

# 2. Load Dataset
try:
    import kagglehub
    path = kagglehub.dataset_download("blastchar/telco-customer-churn")
    data = pd.read_csv(f"{path}/WA_Fn-UseC_-Telco-Customer-Churn.csv")
    print("✅ Dataset loaded from KaggleHub")
except Exception as e:
    print("⚠️ KaggleHub download failed, loading local CSV if available.")
    data = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")

print(data.head())

# 3. Data Preprocessing
# Drop customerID (not predictive)
data = data.drop("customerID", axis=1)

# Convert TotalCharges to numeric (handle errors)
data["TotalCharges"] = pd.to_numeric(data["TotalCharges"], errors="coerce")
data = data.dropna()

# Encode categorical variables
for col in data.select_dtypes(include=["object"]).columns:
    if col != "Churn":
        le = LabelEncoder()
        data[col] = le.fit_transform(data[col])

# Encode target variable
data["Churn"] = data["Churn"].map({"Yes": 1, "No": 0})

# Features and target
```

```
# Features and target
X = data.drop("Churn", axis=1)
y = data["Churn"]

# Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)

# 4. Build Neural Network
model = keras.Sequential([
    Dense(64, activation="relu", input_shape=(X_train.shape[1],)),
    Dropout(0.3),
    Dense(32, activation="relu"),
    Dropout(0.3),
    Dense(1, activation="sigmoid")  # binary classification
])

# Compile model
model.compile(optimizer="adam",
              loss="binary_crossentropy",
              metrics=["accuracy"])

# 5. Train Model
history = model.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=50,
    batch_size=32,
    verbose=1
)

# 6. Evaluate on Test Data
y_pred_prob = model.predict(X_test)
y_pred = (y_pred_prob > 0.5).astype(int)
```

```
print("✅ Test Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# 7. Confusion Matrix
plt.figure(figsize=(6,4))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt="d", cmap="Blues",
            xticklabels=["No Churn", "Churn"], yticklabels=["No Churn", "Churn"])
plt.title("Confusion Matrix - Neural Network")
plt.ylabel("True Label")
plt.xlabel("Predicted Label")
plt.show()

# 8. Training Curves
plt.figure(figsize=(8,5))
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title("Training vs Validation Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

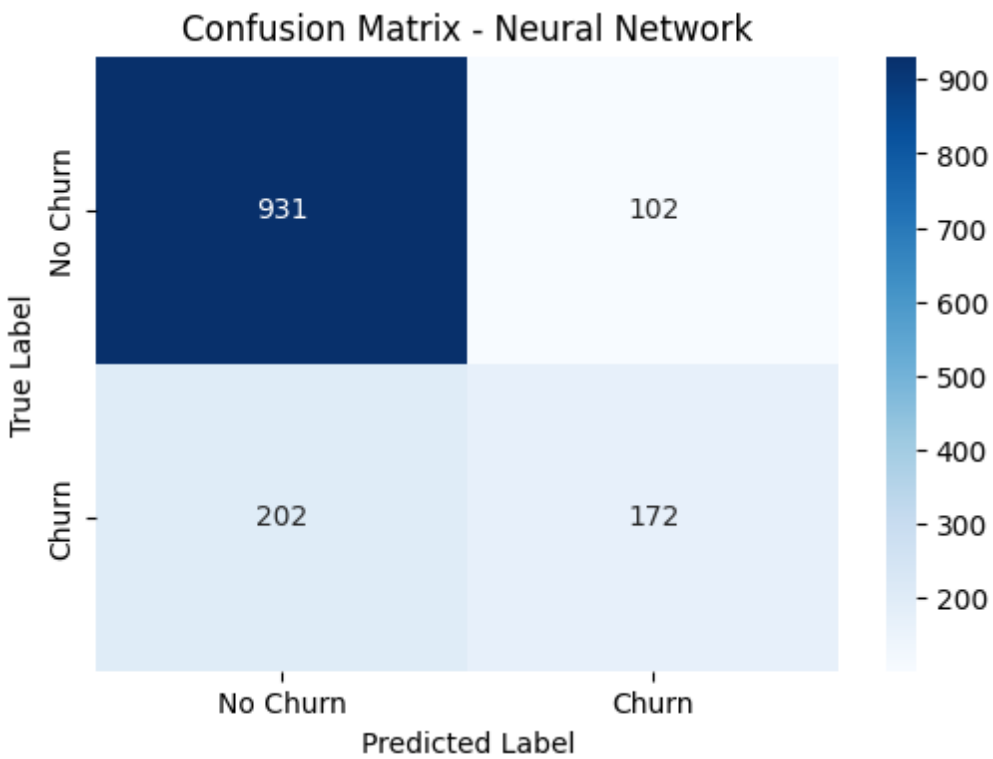
```
Using Colab cache for faster access to the 'telco-customer-churn' dataset.
✅ Dataset loaded from KaggleHub
   customerID  gender  SeniorCitizen Partner Dependents  tenure PhoneService  \
0  7590-VHVEG  Female              0     Yes         No       1           No
1  5575-GNVDE    Male              0      No         No      34          Yes
2  3668-QPYBK    Male              0      No         No       2          Yes
3  7795-CFOCW    Male              0      No         No      45           No
4  9237-HQITU  Female              0      No         No       2          Yes

      MultipleLines InternetService OnlineSecurity  ... DeviceProtection  \
0  No phone service             DSL             No  ...               No
1                No             DSL            Yes  ...              Yes
2                No             DSL            Yes  ...               No
3  No phone service             DSL            Yes  ...              Yes
4                No     Fiber optic             No  ...               No

  TechSupport StreamingTV StreamingMovies        Contract PaperlessBilling  \
0          No          No              No  Month-to-month              Yes
1          No          No              No        One year               No
2          No          No              No  Month-to-month              Yes
3         Yes          No              No        One year               No
4          No          No              No  Month-to-month              Yes

               PaymentMethod MonthlyCharges  TotalCharges Churn
0           Electronic check          29.85         29.85    No
1               Mailed check          56.95        1889.5    No
2               Mailed check          53.85        108.15   Yes
3  Bank transfer (automatic)          42.30       1840.75    No
4           Electronic check          70.70        151.65   Yes

[5 rows x 21 columns]
Epoch 1/50
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shap
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
141/141 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - accuracy: 0.6287 - loss: 0.6188 - val_accuracy: 0.7911 - val_loss: 0.42
Epoch 2/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.7855 - loss: 0.4591 - val_accuracy: 0.8000 - val_loss: 0.41
Epoch 3/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.7799 - loss: 0.4458 - val_accuracy: 0.7991 - val_loss: 0.41
Epoch 4/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.7939 - loss: 0.4355 - val_accuracy: 0.8036 - val_loss: 0.41
Epoch 5/50
```

```
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.7905 - loss: 0.4386 - val_accuracy: 0.8053 - val_loss: 0.40
Epoch 6/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.7962 - loss: 0.4455 - val_accuracy: 0.8062 - val_loss: 0.40
Epoch 7/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.7944 - loss: 0.4378 - val_accuracy: 0.8053 - val_loss: 0.40
Epoch 8/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8088 - loss: 0.4121 - val_accuracy: 0.8044 - val_loss: 0.41
Epoch 9/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8010 - loss: 0.4165 - val_accuracy: 0.8071 - val_loss: 0.41
Epoch 10/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.7980 - loss: 0.4220 - val_accuracy: 0.7929 - val_loss: 0.41
Epoch 11/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.7973 - loss: 0.4280 - val_accuracy: 0.8036 - val_loss: 0.41
Epoch 12/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8079 - loss: 0.4156 - val_accuracy: 0.8044 - val_loss: 0.41
Epoch 13/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8044 - loss: 0.4138 - val_accuracy: 0.8053 - val_loss: 0.41
Epoch 14/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8005 - loss: 0.4224 - val_accuracy: 0.7982 - val_loss: 0.41
Epoch 15/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8127 - loss: 0.4153 - val_accuracy: 0.8009 - val_loss: 0.41
Epoch 16/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8011 - loss: 0.4120 - val_accuracy: 0.8000 - val_loss: 0.41
Epoch 17/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8054 - loss: 0.4141 - val_accuracy: 0.8009 - val_loss: 0.41
Epoch 18/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8196 - loss: 0.3958 - val_accuracy: 0.8009 - val_loss: 0.41
Epoch 19/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8113 - loss: 0.4159 - val_accuracy: 0.7947 - val_loss: 0.41
Epoch 20/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8067 - loss: 0.4096 - val_accuracy: 0.7920 - val_loss: 0.41
Epoch 21/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8072 - loss: 0.4159 - val_accuracy: 0.7964 - val_loss: 0.41
Epoch 22/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8131 - loss: 0.4057 - val_accuracy: 0.7947 - val_loss: 0.42
Epoch 23/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8109 - loss: 0.4121 - val_accuracy: 0.7920 - val_loss: 0.41
Epoch 24/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8180 - loss: 0.3909 - val_accuracy: 0.7956 - val_loss: 0.41
Epoch 25/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8122 - loss: 0.4002 - val_accuracy: 0.7964 - val_loss: 0.41
Epoch 26/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8096 - loss: 0.4071 - val_accuracy: 0.7973 - val_loss: 0.41
```

```
Epoch 27/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8184 - loss: 0.3986 - val_accuracy: 0.7947 - val_loss: 0.41
Epoch 28/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8180 - loss: 0.3990 - val_accuracy: 0.7982 - val_loss: 0.41
Epoch 29/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8120 - loss: 0.4007 - val_accuracy: 0.7973 - val_loss: 0.41
Epoch 30/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8107 - loss: 0.3964 - val_accuracy: 0.7884 - val_loss: 0.41
Epoch 31/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8153 - loss: 0.4108 - val_accuracy: 0.7956 - val_loss: 0.41
Epoch 32/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8162 - loss: 0.4015 - val_accuracy: 0.7956 - val_loss: 0.42
Epoch 33/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8143 - loss: 0.4108 - val_accuracy: 0.7964 - val_loss: 0.42
Epoch 34/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8166 - loss: 0.3951 - val_accuracy: 0.7911 - val_loss: 0.42
Epoch 35/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8146 - loss: 0.4030 - val_accuracy: 0.7964 - val_loss: 0.41
Epoch 36/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8166 - loss: 0.3950 - val_accuracy: 0.7929 - val_loss: 0.42
Epoch 37/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8193 - loss: 0.3972 - val_accuracy: 0.7947 - val_loss: 0.41
Epoch 38/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8102 - loss: 0.4000 - val_accuracy: 0.7929 - val_loss: 0.42
Epoch 39/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8134 - loss: 0.4097 - val_accuracy: 0.7929 - val_loss: 0.42
Epoch 40/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8310 - loss: 0.3801 - val_accuracy: 0.7947 - val_loss: 0.42
Epoch 41/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8235 - loss: 0.3919 - val_accuracy: 0.7929 - val_loss: 0.42
Epoch 42/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8272 - loss: 0.3861 - val_accuracy: 0.7964 - val_loss: 0.42
Epoch 43/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8193 - loss: 0.3959 - val_accuracy: 0.7956 - val_loss: 0.42
Epoch 44/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8244 - loss: 0.3904 - val_accuracy: 0.7929 - val_loss: 0.42
Epoch 45/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8216 - loss: 0.3879 - val_accuracy: 0.7956 - val_loss: 0.42
Epoch 46/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8228 - loss: 0.4006 - val_accuracy: 0.7902 - val_loss: 0.42
Epoch 47/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8245 - loss: 0.3874 - val_accuracy: 0.7911 - val_loss: 0.42
Epoch 48/50
```

```
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8269 - loss: 0.3765 - val_accuracy: 0.7884 - val_loss: 0.42
Epoch 49/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8255 - loss: 0.3887 - val_accuracy: 0.7964 - val_loss: 0.42
Epoch 50/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8351 - loss: 0.3682 - val_accuracy: 0.7929 - val_loss: 0.42
44/44 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
✅ Test Accuracy: 0.7839374555792467
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.82      0.90      0.86      1033
           1       0.63      0.46      0.53       374

    accuracy                           0.78      1407
   macro avg       0.72      0.68      0.70      1407
weighted avg       0.77      0.78      0.77      1407
```



```
# Telco Customer Churn Prediction with Neural Network
# Includes explicit Hyperparameters

# 1. Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.layers import Dense, Dropout

# ---------------------------------------------------
# 🔑 Define Key Hyperparameters
# ---------------------------------------------------
HIDDEN_UNITS_1 = 64          # Number of neurons in first hidden layer
HIDDEN_UNITS_2 = 32          # Number of neurons in second hidden layer
DROPOUT_RATE = 0.3           # Dropout to prevent overfitting
LEARNING_RATE = 0.001        # Adam learning rate
EPOCHS = 50                  # Number of passes through training data
BATCH_SIZE = 32              # Mini-batch size
ACTIVATION_HIDDEN = "relu"   # Activation for hidden layers
ACTIVATION_OUTPUT = "sigmoid" # Binary classification
LOSS_FN = "binary_crossentropy"
OPTIMIZER = keras.optimizers.Adam(learning_rate=LEARNING_RATE)
# ---------------------------------------------------

# 2. Load Dataset
try:
    import kagglehub
    path = kagglehub.dataset_download("blastchar/telco-customer-churn")
    data = pd.read_csv(f"{path}/WA_Fn-UseC_-Telco-Customer-Churn.csv")
    print("✅ Dataset loaded from KaggleHub")
```

```
except Exception as e:
    print("⚠️ KaggleHub download failed, loading local CSV if available.")
    data = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")

print(data.head())

# 3. Data Preprocessing
data = data.drop("customerID", axis=1)
data["TotalCharges"] = pd.to_numeric(data["TotalCharges"], errors="coerce")
data = data.dropna()

for col in data.select_dtypes(include=["object"]).columns:
    if col != "Churn":
        le = LabelEncoder()
        data[col] = le.fit_transform(data[col])

data["Churn"] = data["Churn"].map({"Yes": 1, "No": 0})

X = data.drop("Churn", axis=1)
y = data["Churn"]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)

# 4. Build Neural Network (using hyperparameters)
model = keras.Sequential([
    Dense(HIDDEN_UNITS_1, activation=ACTIVATION_HIDDEN, input_shape=(X_train.shape[1],)),
    Dropout(DROPOUT_RATE),
    Dense(HIDDEN_UNITS_2, activation=ACTIVATION_HIDDEN),
    Dropout(DROPOUT_RATE),
    Dense(1, activation=ACTIVATION_OUTPUT)
])

# Compile model with chosen hyperparameters
model.compile(optimizer=OPTIMIZER,
```

```
                loss=LOSS_FN,
                metrics=["accuracy"])

# 5. Train Model
history = model.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    verbose=1
)

# 6. Evaluate on Test Data
y_pred_prob = model.predict(X_test)
y_pred = (y_pred_prob > 0.5).astype(int)

print("✅ Test Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# 7. Confusion Matrix
plt.figure(figsize=(6,4))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt="d", cmap="Blues",
            xticklabels=["No Churn", "Churn"], yticklabels=["No Churn", "Churn"])
plt.title("Confusion Matrix - Neural Network")
plt.ylabel("True Label")
plt.xlabel("Predicted Label")
plt.show()

# 8. Training Curves
plt.figure(figsize=(8,5))
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title("Training vs Validation Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

```
Using Colab cache for faster access to the 'telco-customer-churn' dataset.
✅ Dataset loaded from KaggleHub
   customerID  gender  SeniorCitizen Partner Dependents  tenure PhoneService  \
0  7590-VHVEG  Female              0     Yes         No       1           No
1  5575-GNVDE    Male              0      No         No      34          Yes
2  3668-QPYBK    Male              0      No         No       2          Yes
3  7795-CFOCW    Male              0      No         No      45           No
4  9237-HQITU  Female              0      No         No       2          Yes

      MultipleLines InternetService OnlineSecurity  ... DeviceProtection  \
0  No phone service             DSL             No  ...               No
1                No             DSL            Yes  ...              Yes
2                No             DSL            Yes  ...               No
3  No phone service             DSL            Yes  ...              Yes
4                No     Fiber optic             No  ...               No

  TechSupport StreamingTV StreamingMovies        Contract PaperlessBilling  \
0          No          No              No  Month-to-month              Yes
1          No          No              No        One year               No
2          No          No              No  Month-to-month              Yes
3         Yes          No              No        One year               No
4          No          No              No  Month-to-month              Yes

               PaymentMethod MonthlyCharges  TotalCharges Churn
0           Electronic check          29.85         29.85    No
1               Mailed check          56.95        1889.5    No
2               Mailed check          53.85        108.15   Yes
3  Bank transfer (automatic)          42.30       1840.75    No
4           Electronic check          70.70        151.65   Yes

[5 rows x 21 columns]
Epoch 1/50
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shap
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
141/141 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.6438 - loss: 0.6262 - val_accuracy: 0.7902 - val_loss: 0.42
Epoch 2/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.7822 - loss: 0.4608 - val_accuracy: 0.7973 - val_loss: 0.41
Epoch 3/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.7831 - loss: 0.4422 - val_accuracy: 0.8062 - val_loss: 0.41
Epoch 4/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.7900 - loss: 0.4371 - val_accuracy: 0.8036 - val_loss: 0.41
Epoch 5/50
```

```
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.7947 - loss: 0.4364 - val_accuracy: 0.8080 - val_loss: 0.41
Epoch 6/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.7834 - loss: 0.4589 - val_accuracy: 0.8018 - val_loss: 0.41
Epoch 7/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8068 - loss: 0.4241 - val_accuracy: 0.8080 - val_loss: 0.40
Epoch 8/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8052 - loss: 0.4199 - val_accuracy: 0.8107 - val_loss: 0.41
Epoch 9/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8063 - loss: 0.4207 - val_accuracy: 0.8018 - val_loss: 0.41
Epoch 10/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8032 - loss: 0.4155 - val_accuracy: 0.8027 - val_loss: 0.41
Epoch 11/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8049 - loss: 0.4156 - val_accuracy: 0.7991 - val_loss: 0.41
Epoch 12/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8027 - loss: 0.4108 - val_accuracy: 0.8018 - val_loss: 0.41
Epoch 13/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8043 - loss: 0.4224 - val_accuracy: 0.8053 - val_loss: 0.41
Epoch 14/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8075 - loss: 0.4114 - val_accuracy: 0.7964 - val_loss: 0.41
Epoch 15/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8063 - loss: 0.4158 - val_accuracy: 0.8080 - val_loss: 0.41
Epoch 16/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8027 - loss: 0.4228 - val_accuracy: 0.7982 - val_loss: 0.41
Epoch 17/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8029 - loss: 0.4092 - val_accuracy: 0.7920 - val_loss: 0.41
Epoch 18/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8118 - loss: 0.4116 - val_accuracy: 0.7938 - val_loss: 0.41
Epoch 19/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8127 - loss: 0.4050 - val_accuracy: 0.7947 - val_loss: 0.41
Epoch 20/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.7980 - loss: 0.4248 - val_accuracy: 0.7876 - val_loss: 0.41
Epoch 21/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8161 - loss: 0.4010 - val_accuracy: 0.7956 - val_loss: 0.41
Epoch 22/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8190 - loss: 0.4082 - val_accuracy: 0.7973 - val_loss: 0.41
Epoch 23/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8083 - loss: 0.4097 - val_accuracy: 0.8027 - val_loss: 0.41
Epoch 24/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8123 - loss: 0.4099 - val_accuracy: 0.8009 - val_loss: 0.41
Epoch 25/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8192 - loss: 0.4123 - val_accuracy: 0.8009 - val_loss: 0.41
Epoch 26/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8075 - loss: 0.4177 - val_accuracy: 0.8027 - val_loss: 0.41
```